

When Abstraction Becomes Indirection: What an AI Coding Agent Must Read

Vasili Gavrilov

Abstract—Classes, interfaces and frameworks were built for human working memory. An AI coding agent reads under a different limit: the context it carries and the relationships it must resolve before acting. We measure that reading as tokens-to-trace, the token count of the smallest set of source regions that decides one endpoint’s behavior, and where each region lives: in the repository, in library source, or in no file at all. Measured on four real systems and tested in 320 controlled agent runs, the plain styles are cheaper: plain Java with JDBC keeps a whole production path in 7.6 thousand tokens inside the project where Spring Boot needs 51 to 124 thousand outside it, and when one endpoint was built on both Java stacks the framework side cost 1.25 turns more per run and gave the only wrong answers, both from a naming convention no source file states.

I. THE QUESTION

A class hides a representation and an interface hides an implementation. A framework hides something larger: control flow. Each is a remedy for a limit of human working memory. An agent is not under that limit and is under others: the context window it fills and the relationships it must resolve before it can act. A hidden implementation is a relationship to resolve, which abstraction adds rather than removes.

The machine-cost objection to abstraction was answered decades ago by compiler work and the zero-overhead principle, and nothing here rests on machine cost. The reader’s cost was never priced, and the reader has changed.

So we ask a narrow question with a countable answer: before an agent can change one endpoint safely, how much source text decides what that endpoint does, and where does that text live?

II. THE MEASURE

Tokens-to-trace of an entry point is the token count of the minimal set of source regions that fully determines its behavior, wire to storage and back: routing, authentication, authorization, handler logic, query, schema, serialization, error mapping. A region is in the set iff removing it leaves some behavior undetermined. The platform floor (JDK, libc, servlet container, database engine, vendored crypto) is excluded identically everywhere; anything the application configures, or that varies by library version, is included. Each region is classified by residence:

- 1) *repository*: the project’s own source;
- 2) *library source*: text outside the repository that decides behavior before the program runs;

- 3) *convention*: fixed by defaults of the exact version in use, recoverable from documentation or a runtime observation, not from any source file.

Tokens are o200k counted with tiktoken [1], and every manifest ships beside the archived paper, so two systems compare by number and a disagreement is about a manifest row, not an interpretation.

III. FOUR SYSTEMS, ONE PATH

We measured one read path in four real systems, each doing the same kind of work: an authenticated or filtered list over a local store.

System	Cat. 1	Cat. 2	Cat. 3
ais, plain C (path only)	5,750	0	0
ais, with its storage engine	14,444	0	0
Monocypher, third-party C (AEAD path)	7,560	0	0
System A, plain Java + JDBC	7,566	0	0
taskwarrior, idiomatic C++ (whole files)	256,664	0	0
Spring Boot 4 RealWorld app	8,175	123,794	>0

TABLE I
O200K TOKENS BY RESIDENCE CATEGORY. CAT. 1 IN-REPOSITORY, CAT. 2 FRAMEWORK SOURCE, CAT. 3 VERSION CONVENTION, NO SOURCE FILE.

Plain Java with JDBC. System A, a commercial web application in production, 39,677 code lines, reported anonymously. The endpoint (token authentication, role gate, per-object ACL, a 4-table JOIN, streamed JSON, central error mapping) closes in 813 lines across 17 files, 7,566 tokens: 3.8% of a 200k window. Every callee is named literally at its call site, so each hop is findable by one search. Categories 2 and 3 measure zero.

Framework Java. A well-regarded, conventional Spring Boot 4 implementation of the RealWorld specification [2]. The matching endpoint needs 8,175 tokens of its own code, 8% more than System A’s whole closure for a smaller domain, plus the 27 framework classes on its dispatch path: 123,794 tokens measured from the published sources jars of the exact BOM-resolved versions (51,342 under a deliberately generous floor). And part of the behavior, the query derived from a repository method name, the security defaults, the filter order, is written in no repository at all: category 3, the version’s conventions.

Plain C. ais, an associative index of 10,597 code lines written to an explicit MISRA-inspired style guide [3]. The recall path closes in 489 lines across 6 files, 5,750 tokens with the storage engine as floor, and 14,444 with the hand-written engine included, wire to bytes on disk: 7% of one window. To bound how much of that is the author rather than the style, the

same rule applied to a library nobody here wrote, Monocypher [4], the audited cipher ais vendors, gives 7,560 tokens for the whole AEAD encrypt path. At the algorithmic extreme the plain closure is the same few thousand tokens whoever writes it.

Idiomatic C++. taskwarrior 2.6.2 [5], the last all-C++ release of a tool developed since 2006, whose `task list` filters records from a local store and prints them. Its closure at whole-file granularity is 99 files, 256,664 tokens, with 43 virtual declarations and 26 overrides on the path. Compared at the same granularity the C closure is 90k against the C++ 257k: 2.8 times the tokens and 6 times the files. One system does not establish the idiom’s cost, and the feature scopes differ; the controlled experiment below is what separates the constructs from the scope.

IV. WHAT THE AGENT PAYS: 280 PAIRED RUNS

Token counts are the static half. The dynamic half is what an agent actually spends, so we built seven pairs: a small C program and the same behavior in C++ through one construct (a virtual registry, a template, operator overloads, RAII, inheritance, a mix, and a control), four maintenance tasks per pair with identical text, five repetitions, 280 runs of one model (claude-sonnet-5, headless), graded by hidden tests the agent never saw. Predictions were recorded before any pair existed.

278 of 280 runs passed, so correctness does not separate the sides at this size. Turn counts do: the C++ side cost more turns in all seven pairs, about 20% on average, medians 8 against 9. Held to identical behavior the C++ text is only 1.17 times the C text, so the cost is not volume. The control pair located it: the same registry classes cost +3.50 turns when spread one-class-per-file over eight files and +0.50 when placed in one file. The agent pays for the walk across files, not for the virtual dispatch. That was not our prediction; the recorded predictions said C++ would be cheaper on local tasks and would break more, and both were wrong.

V. ONE ENDPOINT, TWO JAVA STACKS

The decisive test for a working team is neither C nor constructs; it is the stack choice. So we built one endpoint of the System A kind twice with the same routes, schema, seed rows and observable behavior: an item list and an item create behind bearer-token authentication over a local SQL store. The plain side is one file, JDK `HttpServer` and `JDBC`, 108 lines. The framework side is Spring Boot 3.3 with Spring Data JPA, seven classes in the current idiom: a `OncePerRequestFilter` for the token, a repository interface with derived queries, a `@RestController`. Equivalence at the HTTP boundary was verified over eight probe cases of the contract before any run; outside the contract the sides diverge on a POST body missing a field, 400 from plain’s visible validation against 500 from Spring’s database constraint, a case no task touches. Four maintenance tasks with the same text on both sides, five repetitions, 40 runs, graded by hidden HTTP checks against a live server. The criterion was recorded before the build: the

hypothesis fails if the framework tasks do not cost more than the plain ones.

Task	plain	Spring	Δ	pass	
add a response field	6.6	8.4	+1.8	5/5	5/5
add a query filter	11.0	12.0	+1.0	5/5	3/5
change the 401 contract	6.0	4.2	-1.8	5/5	5/5
find an injected defect	5.0	9.0	+4.0	5/5	5/5

TABLE II
MEAN TURNS PER TASK, FIVE REPETITIONS EACH; PASS COLUMNS PLAIN THEN SPRING.

The framework side cost 1.25 turns more per run (7.15 against 8.40) and 5.3 seconds more wall time; with task as the blocking factor both stand above noise ($p = 0.023$ and $p = 0.022$, permutation within task).

Correctness separates the sides in kind rather than in count, and the way it happened is the finding a practitioner should carry away. Plain passed 20 of 20. Spring passed 18 of 20, and both failures are the same miss: the agent added `@RequestParam(required = false)` `Integer minLen` and correct JPQL behind it, but Spring binds a query parameter by the Java parameter name (Boot builds compile with `-parameters`; without that flag the same code fails loudly instead), the URL said `min_len`, so the parameter stayed null and the endpoint returned the unfiltered list with status 200. The three passing repetitions wrote `@RequestParam(name = "min_len")` explicitly. Nothing in the compiler, the runtime or the response flagged the two that did not: the deciding rule is the category-3 binding convention, and this is what its failure looks like, silent wrong rows. The plain side parses its query string in visible code, so the same mistake had no place to hide, five of five.

The direction is not uniform and the exception is instructive: on the 401-contract task Spring was cheaper, 4.2 turns against 6.0, because the framework’s filter puts the whole authentication contract in one small class while the plain side’s is inline in a longer file. Where the framework genuinely localizes a change, it wins the turn count; where the behavior crosses its conventions, it loses turns or correctness.

VI. THE CONTEXT NOBODY COUNTS

One cost sits outside every table: what a reader must know before reading. For C it is the K&R book, 272 pages, unchanged since 1988. For C++ it is the Stroustrup, 1,368 pages. For a framework it is versioned documentation with no fixed size: the derived-query grammar, the defaults, the filter order all move with the release, and the text that decides them is the text the agent’s training may or may not have current. The two wrong answers above are what that looks like in practice. Old languages with stable semantics give an agent a fixed, finished manual; frameworks give it a moving one.

VII. THREATS, BRIEFLY

All runs sat far below the context window, so what a closure that outgrows the window costs is untested here; the

cross-module volume experiment is named in the archived roadmap. Spring is in every training corpus, so the endpoint runs price recalling its conventions rather than reading them; recall is where its two failures happened. The wall-time gap folds in the slower build the framework brings. One model ran, on one date: agent cost is a property of the model and the representation together. In the pairs, file count changes together with the construct, which is why the control pair exists. And the C++ closures are upper bounds: a human would skim where our rule counts.

VIII. WHAT WE RECOMMEND

Scoped to read cost and to teams working with agents. State the style bias in the prompt: “plain C”, “plain Java with JDBC, streaming”. One such line gives the agent the whole searchable truth of a path for a few thousand tokens, where the layered styles fill the window or depend on text no repository supplies. Keep abstraction where a platform floor earns it, and where it genuinely localizes change, as the 401 task showed; the framework’s win there is real and small, its losses are larger and one of them is silent. Spend the human effort on the objective and the guards: the requirements, and what the tests must assert [6]. And when choosing a stack, measure the closure of one representative path before arguing about taste. The measure is a count; anyone can recompute it.

REFERENCES

- [1] OpenAI, tiktoken. github.com/openai/tiktoken.
- [2] raepd, realworld-springboot-java v2.1.1. github.com/raepd/realworld-springboot-java.
- [3] V. Gavrilov, ais: an associative index. github.com/Anode1/ais.
- [4] L. Vaillant, Monocypher 4.0.3. monocypher.org.
- [5] taskwarrior 2.6.2. github.com/GothenburgBitFactory/taskwarrior.
- [6] V. Gavrilov, agent-recipes. github.com/Anode1/agent-recipes.